

MÓDULO IV

Configuração do Ambiente e Conceitos Básicos em PHP

MÓDULO IV

Como executar um arquivo PHP no navegador

Ao contrário de **HTML** e **JavaScript**, que podem ser executados diretamente no navegador, arquivos **PHP** precisam ser processados por um servidor antes de exibir os resultados.

Para rodar um arquivo **PHP** corretamente, você precisará de um servidor local, como o **Apache**, ou um ambiente como **XAMPP**, **WAMP** ou **MAMP**.

MÓDULO IV

Configurando um servidor local

Se você ainda não tem um ambiente configurado, a maneira mais fácil de rodar **PHP** no navegador é instalando um pacote como **XAMPP**:

1. Faça o download do **XAMPP** em <https://www.apachefriends.org> e instale.
2. Inicie o **Apache** através do painel de controle do **XAMPP**.
3. Coloque seu arquivo **PHP** dentro da pasta **htdocs**, localizada dentro do diretório onde o **XAMPP** foi instalado.

MÓDULO IV

Criando e rodando um arquivo PHP no navegador

Agora que seu servidor está rodando, crie um arquivo chamado **teste.php** dentro de **htdocs** com o seguinte código:

```
<?php  
    echo "Ola, este é meu primeiro script PHP!";  
?>
```

Para executá-lo, abra o navegador e acesse:

`http://localhost/teste.php`

Se tudo estiver correto, você verá a mensagem *"Olá, este é meu primeiro script **PHP!**"* na tela.

MÓDULO IV

Rodando PHP sem XAMPP (Usando o servidor embutido)

Caso não queira instalar um pacote como **XAMPP**, você pode usar o servidor embutido do **PHP**.

Navegue até a pasta onde seu arquivo **PHP** está salvo e execute este comando no terminal:

```
php -S localhost:8000
```

```
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Users\pw001\OneDrive\Área de Trabalho\Nova pasta> php -S localhost:8000
[Thu Oct 23 12:28:17 2025] PHP 8.4.14 Development Server (http://localhost:8000) started
```

Agora, abra o navegador e acesse `http://localhost:8000/teste.php` para ver o resultado.

Executando PHP no terminal!

MÓDULO IV

Por que o PHP precisa de um servidor para rodar no navegador?

Executar arquivos **PHP** no navegador é um passo essencial para quem deseja criar aplicações dinâmicas.

Diferente de linguagens **client-side**, como **JavaScript**, o **PHP** exige um ambiente servidor para ser interpretado antes de exibir seu conteúdo. Essa característica permite que o **PHP** interaja com bancos de dados, processe formulários e personalize páginas conforme as necessidades do usuário.

Por isso, entender como configurar e utilizar um servidor local é um dos primeiros desafios para novos desenvolvedores **PHP**.

Algumas aplicações:

- Criação de sites dinâmicos com **PHP** e **MySQL**.
- Desenvolvimento de sistemas de **login** e **autenticação**.
- Processamento de formulários e manipulação de dados do usuário.
- Geração dinâmica de páginas web e personalização de conteúdo.

MÓDULO IV

Dicas para quem está começando

- Use um servidor local como **XAMPP** ou **WAMP** para rodar seus arquivos **PHP** no navegador.
- Se não quiser instalar um servidor, utilize o servidor embutido do **PHP** com **php -S localhost:8000**.
- Coloque seus arquivos na pasta **htdocs** (**XAMPP**) ou no diretório configurado para rodar **PHP**.
- Ao acessar no navegador, certifique-se de digitar *http://localhost/seuarquivo.php* corretamente.
- Se o **PHP** não funcionar, verifique se o **Apache** está rodando corretamente no painel do **XAMPP**.

MÓDULO IV

O que é Apache?

O **Apache** é um **servidor web livre e de código aberto**, amplamente reconhecido por sua **confiança e flexibilidade**. Ele foi criado em **1995** e é mantido pela **Apache Software Foundation**, organização sem fins lucrativos que desenvolve software de código fonte aberto. Nesse sentido, o **Apache HTTP Server** é o seu projeto mais consagrado.

Este servidor web é projetado para operar em diversos sistemas operacionais. Por conta disso, permite que websites e aplicações sejam hospedados e entregues de maneira eficiente e segura pela internet.

A **Apache Software Foundation** promove um ambiente colaborativo onde desenvolvedores de todo o mundo contribuem para o aprimoramento e suporte constante do **Apache**.

MÓDULO IV

Para que o Apache serve?

O **Apache** é responsável por desempenhar um papel crucial como servidor web. Dentre as suas funções, temos hospedar e entregar conteúdo web de maneira eficiente e segura.

Como servidor web, ele interpreta solicitações do protocolo **HTTP** (*Hypertext Transfer Protocol*) vindas de navegadores e outros clientes web. Assim, pode **responder com páginas web, imagens, vídeos e outros recursos conforme solicitado**. Além disso, o **Apache** é compatível com tecnologias como **PHP, Perl, e Python**, permitindo a criação de sites dinâmicos e interativos.

Isso não apenas assegura uma experiência de usuário consistente. Também **entrega aos desenvolvedores e administradores de sistemas a flexibilidade necessária para configurar e personalizar** o ambiente de hospedagem conforme suas necessidades específicas.

Ou seja, o **Apache** não apenas serve como a base tecnológica para milhões de sites em todo o mundo. Ele **também se destaca por ser confiável e adaptável em um cenário digital que não para de crescer**.

MÓDULO IV

Como funciona o Apache?

O **Apache** funciona como um intermediário entre os navegadores dos usuários e os sites hospedados na internet. Quando um navegador solicita uma página web, ele envia uma solicitação **HTTP** para o servidor onde o **Apache** está instalado.

O Apache então **processa essa solicitação, localiza o arquivo ou recurso solicitado e o envia de volta ao navegador como uma resposta HTTP**.

Essa arquitetura modular do **Apache** permite que desenvolvedores adicionem e configurem módulos conforme necessário. Isso maximiza suas funções para suportar diferentes linguagens de programação, segurança avançada e outras personalizações.

MÓDULO IV

Quais as vantagens de utilizar o Apache?

O **Apache** apresenta uma série de **vantagens que o colocam como um dos servidores web mais populares e confiáveis atualmente**. Por ser gratuito e de código aberto, o **Apache** se destaca como uma escolha versátil e econômica para empresas e desenvolvedores.

Vamos conhecer suas principais vantagens:

Preço

O Apache é totalmente **gratuito e de código aberto, o que significa que pode ser utilizado sem custos de licenciamento**. Isso reduz os gastos associados à criação e manutenção de servidores. Além disso, também promove uma comunidade de desenvolvedores ativa que colabora para melhorar continuamente o software.

MÓDULO IV

Compatibilidade

O **Apache** é conhecido por sua compatibilidade abrangente com diferentes sistemas operacionais, como **Linux**, **Windows** e **macOS**. Essa flexibilidade permite que desenvolvedores e administradores escolham o ambiente de servidor que melhor se adapta às suas necessidades específicas.

Sua empresa prefere **Linux** como padrão ou você é um desenvolvedor que usa **macOS**?

O **Apache** oferece uma solução estável e confiável que se integra facilmente a diferentes plataformas, garantindo uma hospedagem web eficiente e consistente.

MÓDULO IV

Recursos

O **Apache** oferece uma variedade de recursos que o tornam uma escolha bem completa para servidores web. Ele suporta múltiplas linguagens de programação, permitindo o desenvolvimento de sites dinâmicos e interativos.

Sua configuração é flexível, permitindo ajustes precisos conforme as necessidades específicas de cada aplicação. Além disso, o **Apache** inclui suporte integrado para **SSL/TLS**, garantindo a segurança das comunicações online por meio de criptografia robusta.

MÓDULO IV

Suporte

O **Apache** conta com uma vasta comunidade de usuários e desenvolvedores que contribuem para seu suporte e evolução contínua. Essa comunidade ativa oferece uma extensa documentação, tutoriais detalhados e fóruns de ajuda. Por sua vez, isso facilita o suporte técnico e a resolução de problemas para aqueles que utilizam o **Apache** em seus projetos.

Essa colaboração contínua não apenas favorece melhorias constantes no software. Ela também assegura que usuários de todos os níveis de habilidade possam aproveitar ao máximo as capacidades do **Apache**.

MÓDULO IV

Modularidade

O **Apache** também é conhecido por sua arquitetura modular, que oferece uma flexibilidade surpreendente para personalização.

Seus módulos permitem adicionar ou remover recursos conforme necessário, isso permite que se adapte às demandas específicas de cada projeto. Facilita desde a integração de novas tecnologias até a implementação de medidas avançadas de segurança.

A modularidade do **Apache** não apenas simplifica o gerenciamento e a otimização de recursos. Ela também garante que ele possa evoluir com as necessidades em constante mudança do ambiente digital.

MÓDULO IV

Conclusão

O **Apache** se destaca como uma escolha muito eficaz para servidores web, oferecendo uma combinação única de confiabilidade, flexibilidade e custo-benefício.

Como um software de código aberto e gratuito, ele não apenas reduz os custos. Também promove uma comunidade global de desenvolvedores que constantemente melhoram e expandem suas capacidades.

O **Apache** conta com suporte robusto, ampla compatibilidade com sistema operacional e recursos avançados como **SSL/TLS** para segurança. Por isso, ele continua a ser a escolha preferida para milhões de sites ao redor do mundo.

MÓDULO IV

Como configurar Apache Virtual Hosts no Windows

O que é um Virtual Host?

Um **Virtual Host** é, em poucas palavras, um **servidor virtual**. Quando usamos um **Virtual Host**, podemos criar uma representação de um site em nosso servidor **Apache**, mas continuaremos testando códigos locais. Para entendermos melhor, pense no seguinte: quando iniciamos nosso servidor **Apache**, sempre acessamos no navegador o **localhost** ou **127.0.0.1** (*nosso lar doce lar*). Com esse servidor local nós podemos testar e trabalhar em vários projetos. Mas, imagine que precisamos testar dois projetos ao mesmo tempo: um *site de um restaurante* e um *site de notícias*. Para fazer os testes precisaríamos acessar **localhost/restaurante** e **localhost/noticias**, por exemplo. Não seria melhor se pudéssemos já simular como esses projetos ficariam no servidor remoto? Por exemplo: acessar **projeto-restaurante.com.br** e **projeto-noticias.com.br** e testar esses dois projetos de maneira independente? É nessa situação que um **Virtual Host** poderia nos ajudar, pois ele cria domínios diferentes para o mesmo servidor.

MÓDULO IV

Vantagens de usar um Virtual Host

Como vimos, uma das vantagens em usar **Virtual Hosts** é que eles nos dão a possibilidade de testar mais de um projeto ao mesmo tempo em nosso servidor local. Mas, não é apenas isso.

Pense agora no **Servidor Remoto**. Geralmente imaginamos que um servidor pode conter apenas um site.

Contudo, se aplicarmos o conceito de **Virtual Hosts** também em nossa **hospedagem**, poderemos ter mais de um projeto hospedado no mesmo servidor. Isso é incrível, pois poderemos até mesmo economizar os custos com servidores.

Também poderemos testar nossos projetos de maneira mais organizada, em um ambiente mais próximo do dia a dia.

MÓDULO IV

Como criar um Virtual Host

Nesse artigo iremos falar sobre criar Virtual Hosts no Windows, com a ajuda do **XAMPP**.

A primeira coisa que precisamos fazer é identificar a localização do arquivo **httpd-vhosts.conf** em nosso **Apache**. No **XAMPP**, esse arquivo fica na pasta:

C:/xampp/apache/conf/extra/httpd-vhosts.conf.

Ao acessar esse arquivo, poderemos criar nosso primeiro **host virtual**.

MÓDULO IV

A estrutura básica de um Virtual Host é a seguinte:

```
<VirtualHost *:80>  
    ServerName www.meuprojeto.com.br  
    ServerAlias meuprojeto.com.br  
    DocumentRoot "caminho_para_o_projeto"  
</VirtualHost>
```

Explicando: para criar um **Virtual Host** usamos uma estrutura parecida a de uma tag do **HTML**. Informamos qual porta será usada para acessar esse **host**. Definimos a porta **80** pois ela é a padrão para acessar sites na Web, representa o protocolo **HTTP**. Dentro da “tag” do **Virtual Host** nós definimos três configurações.

MÓDULO IV

- 1 - **ServerName**. Representa o nome do servidor, ou seja, o domínio que iremos usar para acessar o projeto.
- 2 - **ServerAlias**. Cria um apelido para o servidor, ou seja, um nome alternativo que irá redirecionar a requisição para o domínio principal, garantindo que o usuário não tenha um erro de caminho não encontrado se informar um host diferente.
- 3 - **DocumentRoot**. Define o caminho dentro de nosso computador para encontrar o projeto de fato.

Essa estrutura funciona na maioria das vezes. Contudo, pode haver alguma situação em que ela não consiga ser executada. Para garantir seu funcionamento, é interessante adicionar o seguinte trecho de código à “tag” do **Virtual Host**:

MÓDULO IV

```
<Directory "caminho para o projeto">  
    Options Indexes FollowSymLinks Includes ExecCGI  
    AllowOverride All  
    Require all granted  
</Directory>
```

Após qualquer alteração neste arquivo, precisamos reiniciar o servidor **Apache** para ver as mudanças sendo refletidas.

Por último, precisamos fazer uma pequena alteração em um arquivo de configuração do nosso Sistema Operacional, o arquivo **hosts**. Isso é necessário pois, quando formos acessar o domínio que criamos, o nosso computador irá procurar por seu endereço na Internet e não irá encontrá-lo ou irá nos redirecionar para um site que não desejamos. Por isso, nós iremos deixar claro para o **Windows** que queremos que ele procure o domínio em nosso próprio servidor.

MÓDULO IV

O arquivo hosts fica armazenado em *C:/Windows/System32/drivers/etc/hosts*.

Esse arquivo contém o seguinte padrão:

o nome do servidor a ser redirecionado e o domínio que estamos desejando redirecionar.

Assim, basta adicionar essa linha:

```
127.0.0.1
```

```
www.meuprojeto.com.br
```

Ao editar este arquivo e tentar salvá-lo, pode ser que o **Windows** não permita, devido à falta de permissão. Nesse caso, recomendamos que você use o **Visual Studio Code** para realizar a edição. Se não for possível utilizá-lo, você pode seguir o seguinte passo a passo: copie o arquivo hosts e cole-o em sua **Área de Trabalho**; edite-o normalmente e salve. Por fim, copie-o novamente e cole na pasta de origem, com o objetivo de substituí-lo. O **Windows** irá pedir permissão de **Administrador**.

Basta concedê-la e então o arquivo estará salvo com as alterações.

Com isso nosso **host virtual** estará funcionando.

MÓDULO IV

Exemplo Prático

Iremos criar um projeto simples de um restaurante em uma pasta e um projeto de um site de notícias em outra. Ambas possuirão um arquivo **index.php**.

O arquivo do projeto do restaurante ficará assim:

```
<?php
```

```
echo "<h2>Hello World! Sejam bem-vindo ao meu Projeto de Restaurante</h2>";
```

E o arquivo do projeto de notícias ficará assim:

```
<?php
```

```
echo "<h2>Hello World! Sejam bem-vindo ao meu Projeto de Notícias</h2>";
```

MÓDULO IV

Agora vamos criar um **Virtual Host** para cada um desses projetos.

Nosso arquivo **httpd-vhosts.conf** ficará dessa forma:

```
<VirtualHost *:80>
    ServerName www.projeto.restaurante.com.br
    ServerAlias projeto.restaurante.com.br
    DocumentRoot "C:\Users\Anthony Rafael\Desktop\restaurante"
    <Directory "C:\Users\Anthony Rafael\Desktop\restaurante">
        Options Indexes FollowSymLinks Includes ExecCGI
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>

<VirtualHost *:80>
    ServerName www.projeto.noticias.com.br
    ServerAlias projeto.noticias.com.br
    DocumentRoot "C:\Users\Anthony Rafael\Desktop\noticias"
    <Directory "C:\Users\Anthony Rafael\Desktop\noticias">
        Options Indexes FollowSymLinks Includes ExecCGI
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

MÓDULO IV

Perceba que o que realmente varia entre um e outro são os caminhos para os projetos e o nome do domínio que estamos criando. Só falta adicionar esses novos domínios ao arquivo **hosts**, que ficará assim:

```
127.0.0.1 www.projeto.restaurante.com.br  
127.0.0.1 www.projeto.noticias.com.br
```

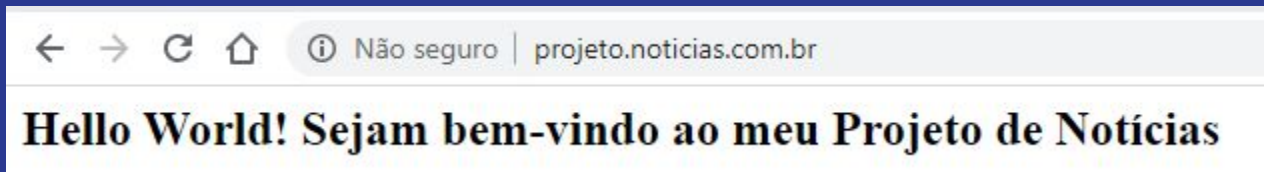
Um detalhe importante: perceba que adicionamos neste arquivo o domínio que está no **ServerName**, por ser o identificador principal de nosso **host**. Depois de realizar essas alterações, basta reiniciar o **Apache**.

Agora, se digitarmos em nosso navegador o endereço *www.projeto.restaurante.com.br*, será retornado o seguinte:



MÓDULO IV

E ao acessar *www.projeto.noticias.com.br* veremos o seguinte na tela:



Excelente! Os dois **Virtual Hosts** estão sendo identificados. O melhor é que os dois servidores estão sendo executados **ao mesmo tempo**. Assim, podemos realizar alterações em um projeto e elas serão refletidas na hora, sem interferir na estrutura ou funcionalidade do outro.

Utilizar **Virtual Hosts** ajuda muito o nosso desenvolvimento.
Faça testes em seu servidor **Apache** e use esse recurso sempre que precisar.

MÓDULO IV

Mas O Que É MySQL?

Uma empresa sueca chamada **MySQL AB** desenvolveu o **MySQL** em **1994**.

Então, a companhia norte-americana *Sun Microsystems* obteve controle total do software ao comprar a **MySQL AB** em **2008**.

Já em **2010**, a gigante **Oracle**, também norte-americana, por sua vez comprou a *Sun Microsystems*, e o **MySQL** tem sido da **Oracle** desde então.

Quanto a sua definição, **MySQL** é um Banco de Dados relacional (**RDBMS** – *Relational Database Management Systems*) com um modelo de **cliente-servidor**.

RDBMS é um software de código aberto ou serviço usado na criação e gerenciamento de bancos de dados baseados no modelo relacional. Agora vamos analisar cada termo.

MÓDULO IV

Banco de Dados

Um **banco de dados** nada mais é do que uma coleção de **dados estruturados**.

Pense em uma **selfie**: você aperta o botão para tirar uma foto de você mesmo.

A foto é o dado, enquanto a galeria do seu celular onde a foto ficou armazenada é o **banco de dados**.

Banco de dados é um local onde dados são armazenados e gerenciados.

A palavra “**relacional**” significa que os dados armazenados estão organizados em **tabelas**.

Cada **tabela** está relacionada de alguma maneira. Caso o software não suporte o modelo relacional, então chamamos **DBMS**.

MÓDULO IV

Código Aberto

Código aberto significa que você tem a liberdade de usar e modificar.

Qualquer pessoa pode instalar o software.

Você também pode aprender sobre o código e modificá-lo para melhor atender suas necessidades. Mas o **GPL** (*GNU – Licença Pública Geral*) determina o que é possível ser feito de acordo com as condições. A versão comercializável da licença também está disponível caso você precise de mais flexibilidade e suporte avançado.

MÓDULO IV

Modelo Cliente-Servidor

Computadores que rodam softwares de **RDBMS** são chamados clientes. Sempre que precisam de dados, eles se conectam a um servidor **RDBMS**.

MySQL é um dos muitos clientes **RDBMS** disponíveis.

RDBMS e **MySQL** são muitas vezes confundidos como sendo a mesma coisa devido a popularidade do **MySQL**.

Algumas aplicações de renome, como **Facebook**, **Twitter**, **YouTube**, **Google** e **Yahoo!** utilizam **MySQL** para o armazenamento de dados.

Mesmo que tenha sido inicialmente criada para uso ilimitado, atualmente é ele compatível com muitas plataformas importantes como **Linux**, **MacOS**, **Microsoft Windows** e **Ubuntu**.

MÓDULO IV

SQL

É também importante saber a diferença entre **SQL** e **MySQL**.

O **MySQL** é uma marca de software **RDBMS** mais populares que utiliza o modelo **cliente-servidor**.

Mas como o cliente e o servidor se comunicam em um ambiente **RDBMS**?

Eles usam uma linguagem específica – **Structured Query Language (SQL)**.

Se você encontrar outros nomes que também possuam **SQL**, como **PostgreSQL** e **Microsoft SQL Server**, provavelmente são outras marcas que utilizam a sintaxe **SQL**.

Um software **RDBMS** é normalmente escrito em outra linguagem de programação, mas sempre utiliza **SQL** como linguagem primária para comunicação com o **banco de dados**.

O próprio **MySQL** é escrito em **C** e **C++**. Pense em todos os países colonizados pela Inglaterra, cada um tem sua história e costumes próprios, mas todos falam inglês.

MÓDULO IV

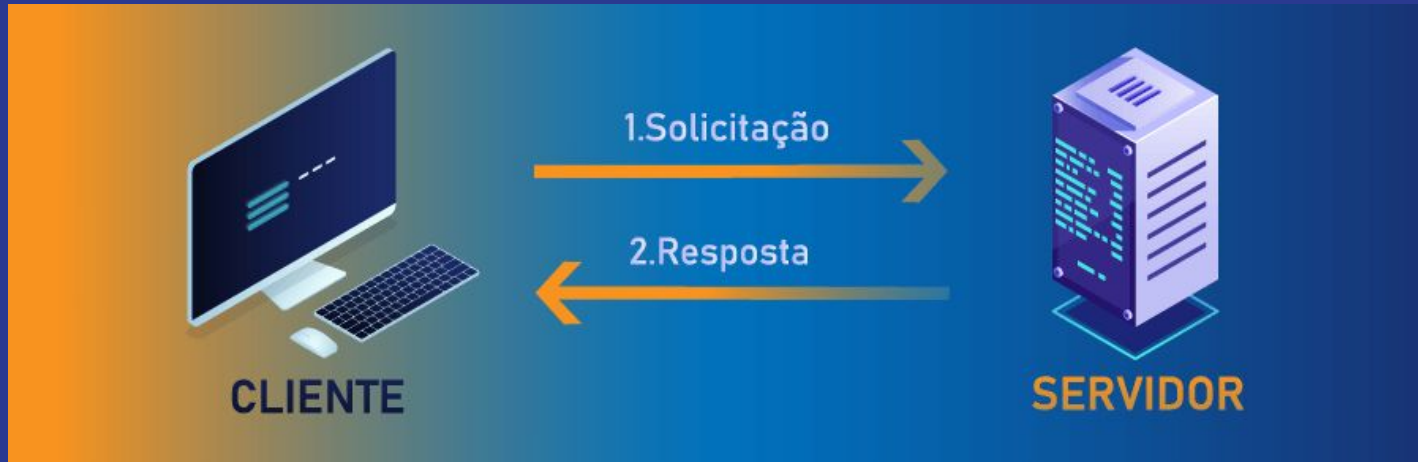
O cientista da computação **Ted Codd** desenvolveu **SQL** no início dos anos **70** com um **IBM** baseado no modelo relacional. A linguagem se tornou mais popular em **1974** substituindo algumas linguagens ultrapassadas, como **ISAM** e **VISAM**.

Colocando a história de lado, o **SQL** informa ao servidor o que deve ser feito com o dado. Ele desempenha as seguintes funções:

- **Data Query.** Solicitar informações específicas de um banco de dados.
- **Data Manipulation.** *Adicionar, excluir, mudar, ordenar* e outras operações para modificar os dados, valores ou visuais.
- **Data Identity.** Definir tipos de dados. Ex: mudar dados numéricos para integrais. Isso também inclui definir a relação das tabelas no banco de dados.
- **Data Access Control.** Fornece técnicas de segurança para proteger dados, incluindo quem pode visualizar ou usar as informações armazenadas no banco de dados.

MÓDULO IV

Como Funciona o MySQL?



MÓDULO IV

A imagem explica a estrutura básica de um **modelo cliente-servidor**.

Um ou mais dispositivos (*clientes*) conectam com o servidor através de uma rede específica.

Cada cliente pode fazer a solicitação através de uma interface gráfica do usuário (*IGU*) em suas telas. E o servidor vai produzir o melhor resultado, desde que ambos entendam as instruções. Sem entrar muito nos méritos técnicos, os principais processos que acontecem em um ambiente **MySQL** são os mesmos.

- O **MySQL** cria um banco de dados para armazenamento e manipulação de dados, definindo a relação de cada tabela.
- Clientes podem fazer solicitações digitando comandos **SQL** específicos no **MySQL**.
- A aplicação do servidor responde com a informação solicitada fazendo aparecer no cliente.

MÓDULO IV

Quanto ao cliente, geralmente se enfatiza qual **IGU MySQL** é utilizado.

Quanto mais leve e simples de usar for o **GUI**, mais eficiente será.

Alguns dos **IGU** mais populares são **MySQL WorkBench**, **SequelPro**, **DBVisualizer** e **Navicat DB Admin Tool**.

Alguns deles são gratuitos, enquanto outros são pagos. Alguns rodam exclusivamente no **MacOS**, enquanto outros são compatíveis com os maiores sistemas operacionais.

Cada cliente deve escolher um **IGU** de acordo com suas necessidades.

Para gerenciamento de bancos de dados web, incluindo sites **WordPress**, a opção mais comum é o **phpMyAdmin**.

MÓDULO IV

Por que o MySQL é tão popular?

341 systems in ranking, December 2018

Rank			DBMS	Database Model	Score		
Dec 2018	Nov 2018	Dec 2017			Dec 2018	Nov 2018	Dec 2017
1.	1.	1.	Oracle +	Relational DBMS	1283.22	-17.89	-58.32
2.	2.	2.	MySQL +	Relational DBMS	1161.25	+1.36	-156.82
3.	3.	3.	Microsoft SQL Server +	Relational DBMS	1040.34	-11.21	-132.14
4.	4.	4.	PostgreSQL +	Relational DBMS	460.64	+20.39	+75.21
5.	5.	5.	MongoDB +	Document store	378.62	+9.14	+47.85
6.	6.	6.	IBM Db2 +	Relational DBMS	180.75	+0.87	-8.83
7.	7.	↑ 8.	Redis +	Key-value store	146.83	+2.66	+23.59
8.	8.	↑ 10.	Elasticsearch +	Search engine	144.70	+1.24	+24.92
9.	9.	↓ 7.	Microsoft Access	Relational DBMS	139.51	+1.08	+13.63
10.	10.	↑ 11.	SQLite +	Relational DBMS	123.02	+0.31	+7.82

MÓDULO IV

O Servidor **MySQL** não é o único **(R)DBMS** no mercado, mas é um dos mais populares, ficando atrás apenas do **Oracle Database**.

E isso apenas em parâmetros importantes, como o número de menções em pesquisas, perfis profissionais no **LinkedIn** e frequência de debates em fóruns técnicos.

O fato de muitas empresas de tecnologia gigantes utilizam seus serviços é um dos aspectos que mais impactam em sua popularidade.

Abaixo veremos algumas das suas principais características.

MÓDULO IV

Flexibilidade e Facilidade de Uso

Você pode modificar o código-fonte para atender suas expectativas e você não precisa pagar nada por isso. Além de ter a opção de upgrade para uma versão comercial. O processo de instalação é relativamente simples, não levando mais que 30 minutos.

Alto Desempenho

Um vasto array de servidores cluster estão disponíveis no **MySQL**. Quer você esteja armazenando uma quantidade gigante de dados e-commerce ou realizando atividades de inteligência pesadas, **MySQL** garante um alto desempenho e velocidade.

MÓDULO IV

Um Padrão da Indústria

O **MySQL** tem sido usado por indústrias há muito tempo, o que significou que há abundância de recursos para desenvolvedores experientes. Sempre que precisar de alguma aplicação não vai ser difícil encontrar desenvolvedores que estejam dispostos a trabalhar com **MySQL**.

Segurança

Seus dados devem ser a principal preocupação ao escolher um software **RDBMS**. Com seu *Sistema de Privilégios de Acesso e Gerenciamento de Contas de Usuários*, **MySQL** possui um alto nível de segurança. Verificação auto-hospedada e criptografia de senhas estão disponíveis.

MÓDULO IV

O que é MySQL ?

MySQL é um sistema de gerenciamento de banco de dados (**SGBD**), gratuito e livre.

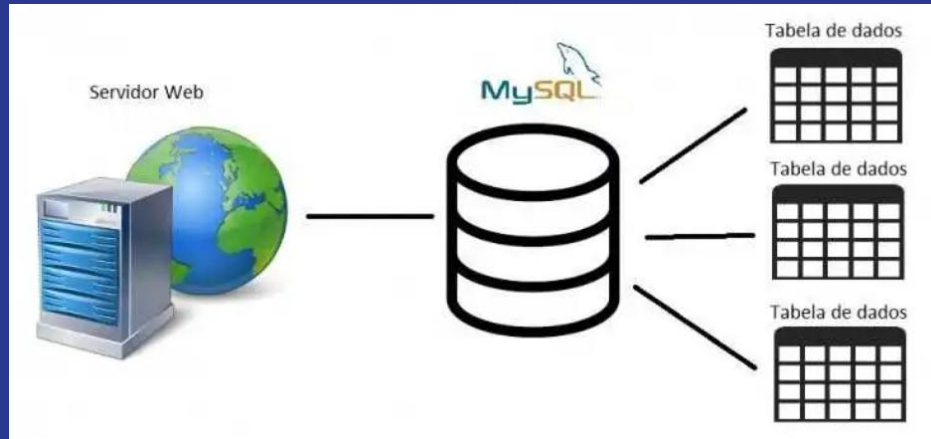
Criado inicialmente em **1995**, foi sofrendo evoluções com o tempo, e atualmente é a plataforma mais utilizada no mundo. Trata-se de um serviço estável, seguro e confiável. Uma ferramenta bastante poderosa. O **MySQL** foi criado por **David Axmark** e **Michael Widenius**, dois engenheiros da Suécia.

Uma pergunta frequente dos nossos clientes é o que é **mysql**, e para que ele serve?.

WordPress, Magento, Woocommerce e Prestashop são exemplos de aplicações que fazem uso do MySQL.

MÓDULO IV

Por exemplo, um servidor web (*por exemplo o **Apache***), acessa o banco de dados **MySQL**. Ele solicita consultas, inserção de dados e exclusão de dados. Por sua vez, o **MySQL** encaminha esses pedidos e faz toda a gestão interna dos dados. Por fim, o **MySQL** devolve ao **servidor web** os dados solicitados.



MÓDULO IV

O que é e como ele armazena os dados ?

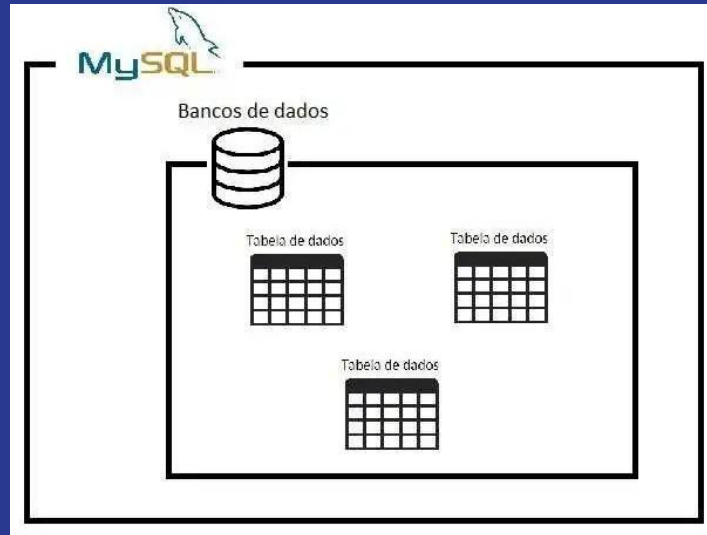
Basicamente, temos a seguinte hierarquia: em primeiro, temos o sistema de gestão **MySQL**. Em segundo lugar, temos os **bancos de dados**. E em terceiro lugar, temos as **tabelas de dados**.

As **tabelas de dados** armazenam, por exemplo, um cadastro dos clientes. **Nome, telefone, email, etc.**
Ou então, armazenam uma lista de produtos de uma loja virtual. **Produto, preço, cor, etc.**

Um **banco de dados** contém um conjunto de tabelas dentro dele. Assim, o **banco de dados** possui *atributos de segurança, controle de acesso, funções de backup, entre outros.*

MÓDULO IV

Dessa forma, o sistema de gestão (**MySQL**), controla todos os bancos de dados. Também, ele controla toda comunicação externa com outros servidores que realizam consultas e operações com os dados.



MÓDULO IV

Quais os comandos de MySQL mais comuns ?

O **MySQL** faz uso de **SQL** (*Structured Query Language*). Trata-se de uma linguagem universal para consultas e operações em banco de dados. Assim, a estrutura de **SQL** é bastante parecida entre os bancos de dados mais comuns (*MySQL, SQL Server, Oracle, Postgree*). Afinal, o que é **MySQL** e o que posso fazer com ele ? A resposta é: ele serve para armazenar os dados de seu site ou sua empresa, de forma segura e organizada.

Os comandos mais comuns de **SQL** são **CONNECT, SELECT, INSERT, UPDATE e DELETE**.

Por eles, podemos realizar *consultas, inserir dados, atualizar e apagar*. Assim, há ainda outros comandos mais avançados, para criação de rotinas automáticas, índices, concessão de permissões, etc.

MÓDULO IV

Existem ferramentas de gerenciamento que facilitam a nossa vida.

Elas permitem gerenciar um **banco de dados MySQL** sem precisar decorar estes comandos.

Dessa forma, elas são o **phpMyAdmin**, que funciona via web, e o **Heidi SQL** e **DBeaver**, que funcionam em desktop.

- **CONNECT** é o comando usado para estabelecer a primeira conexão com o banco de dados. Antes de realizar qualquer consulta, é necessário fazer a conexão. Neste artigo, explicamos como **conectar ao MySQL usando o PHP**.
- **SELECT** é utilizado para realizar uma consulta ao banco de dados. Por uma consulta, podemos escolher quais dados queremos trazer (filtros). Por exemplo, com o **SELECT** podemos consultar clientes cadastrados no banco.
- **INSERT** é usado para inserir dados no banco. Ou seja, criar um novo registro no banco de dados. Por exemplo, com ele podemos inserir o cadastro de um novo cliente.
- **UPDATE** é o comando usado para atualizar dados no banco. Por exemplo, quando queremos alterar o telefone de contato de um cliente cadastrado.
- **DELETE** é usado para apagar dados. Por exemplo, quando quisermos apagar o cadastro de um cliente.

MÓDULO IV

Gerenciamento de dados no MySQL

O **MySQL** é um banco de dados relacional que permite armazenar, gerenciar e consultar dados de forma eficiente. Para trabalhar com dados no **MySQL**, é necessário *criar tabelas, inserir, atualizar e excluir dados*. A seguir, vamos discutir estes conceitos em detalhes. Assim, agora veremos aplicações dos comandos citados anteriormente.

MÓDULO IV

- Criação de tabelas:

Antes de inserir dados no MySQL, é necessário criar uma tabela para armazená-los.

A **criação de uma tabela** envolve definir os nomes das colunas, os seus tipos de dados e as restrições de integridade. Dessa forma, a sintaxe para criar uma tabela no **MySQL** é a seguinte:

```
CREATE TABLE nome da tabela (  
    coluna1 tipo de dado,  
    coluna2 tipo de dado,  
    coluna3 tipo de dado);
```

MÓDULO IV

Por exemplo, a seguir está o código **SQL** para criar uma tabela de clientes:

```
CREATE TABLE clientes (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(50),  
    email VARCHAR(50),  
    telefone VARCHAR(15),  
    endereco VARCHAR(100)  
);
```

MÓDULO IV

- Inserção de dados:

Após criar uma tabela, podemos inserir dados nela usando a instrução **INSERT**.

A seguir, vamos mostrar um exemplo de como inserir **vários tipos dados** na tabela de clientes criada anteriormente:

```
INSERT INTO clientes (nome, email, telefone, endereco)  
VALUES ('João', 'joao@email.com', '1234567890', 'Rua Exemplo 123');
```

MÓDULO IV

- Atualização de dados:

Para atualizar dados em uma tabela, é possível usar a instrução **UPDATE**. Assim, vamos mostrar mostrado um exemplo de como **atualizar** o e-mail de um cliente:

```
UPDATE clientes  
SET email = 'joão@email.com.br'  
WHERE id = 1;
```

MÓDULO IV

- Exclusão de dados:

Para excluir dados de uma tabela, é possível usar a instrução **DELETE**.

A seguir, vamos mostrar um exemplo de como excluir um cliente:

```
DELETE FROM clientes  
WHERE id = 1;
```

Além dessas **instruções básicas**, o **MySQL** oferece muitas outras funções e recursos para gerenciar dados, como a capacidade de *fazer consultas e filtros, criar índices, usar ordenação, funções de agregação* e muito mais.

Dessa forma, o **MySQL** é um banco de dados poderoso e versátil que oferece muitas ferramentas para gerenciar e manipular dados de forma eficiente. Ao dominar as instruções básicas de *criação de tabelas, inserção, atualização e exclusão de dados*, é possível **trabalhar com dados no MySQL de forma eficiente e segura**.

MÓDULO IV

Posso armazenar qualquer tipo de arquivo ?

Não exatamente. O **MySQL** serve para armazenar dados, predominantemente dados texto. Por exemplo, um cadastro de um cliente, um texto, até mesmo todo conteúdo de um livro. Assim, possui a opção de armazenar arquivos binários (*como por exemplo uma imagem*). No entanto, isto causa uma degradação de performance muito grande no banco de dados. Devido ao tamanho, uma música ou vídeo não podem ser armazenados em **MySQL**

MÓDULO IV

Qual é o preço do MySQL ?

É um software **gratuito e livre**. Não possui custo para instalação.
Qualquer um pode instalar o **MySQL** e criar bancos de dados livremente.
Assim, o podemos acessar o **MySQL** remotamente por meio de programas gratuitos.
Atualmente o **MySQL** pertence a **Oracle**.

MÓDULO IV

O que é um Usuário MySQL?

Um usuário **MySQL** é como uma identidade digital que permite que alguém acesse e interaja com um banco de dados **MySQL**, que é um local onde informações são armazenadas de forma organizada. Assim como você precisa de um nome de **usuário** e **senha** para entrar em uma conta online, um usuário **MySQL** precisa de um nome de usuário e uma senha para acessar e realizar operações no banco de dados.

Cada usuário **MySQL** pode ter permissões diferentes, que determinam o que eles podem fazer dentro do banco de dados. Por exemplo, alguns usuários podem ter permissão para ler e escrever dados, enquanto outros podem apenas ler informações. Portanto, um usuário **MySQL** é como uma porta de entrada segura para acessar e manipular dados dentro de um banco de dados **MySQL**.

MÓDULO IV

O que são as permissões de usuário no MySQL?

Ao explorar o **MySQL**, é essencial entender as permissões de usuário, como chaves que abrem portas específicas em um espaço complexo. Abaixo, uma lista das permissões do **MySQL**:

- **SELECT:** Ver dados.
- **INSERT:** Adicionar novos dados.
- **UPDATE:** Modificar dados.
- **DELETE:** Excluir dados.
- **CREATE:** Criar tabelas/bancos.
- **ALTER:** Modificar estruturas.
- **DROP:** Excluir tabelas/bancos.
- **GRANT:** Dar permissões.
- **REVOKE:** Remover permissões.
- **ALL PRIVILEGES:** Todas as permissões.

MÓDULO IV

Como Criar Usuário MySQL e Gerenciar Permissões via Linha de Comando

Atenção! Para realizar esse procedimento é necessário possuir o acesso root do servidor.

Acessando o Terminal

Abra o terminal ou **prompt** de comando no seu sistema, e certifique-se de que você tenha as credenciais de administrador (*como root*) para o **MySQL**.

MÓDULO IV

Conectar ao MySQL

Digite o seguinte comando e pressione Enter:

```
mysql -u root -p
```

Inserir senha

Executar o comando solicitará a senha do usuário **root** do **MySQL**.

Insira a senha para acessar o prompt do **MySQL**.

Criar um Novo Usuário

Para criar um novo usuário, use o comando a seguir:

```
CREATE USER 'nome_do_usuario'@'localhost' IDENTIFIED BY 'senha';
```

MÓDULO IV

Substituir parâmetros

Substitua **nome_do_usuario** pelo nome do usuário desejado e **senha** pela senha que você deseja associar a ele.

Conceder Permissões

Agora, vamos conceder permissões ao novo usuário.

Por exemplo, para conceder permissão para **ler** e **escrever** em um banco de dados chamado **meubanco**, use:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON meubanco.* TO 'nome_do_usuario'@'localhost';
```

MÓDULO IV

Se você quiser conceder todas as permissões para um usuário no banco de dados, utilize o seguinte código:

```
GRANT ALL PRIVILEGES ON meubanco.* TO 'nome_do_usuario'@'localhost';
```

Atualizar Privilégios

Depois de conceder permissões, você precisa atualizar os privilégios para que eles entrem em vigor:

```
FLUSH PRIVILEGES;
```

MÓDULO IV

Verificar Permissões

Para verificar as permissões concedidas a um usuário, você pode usar o comando:

```
SHOW GRANTS FOR 'nome_do_usuario'@'localhost';
```

Sair do MySQL

Quando terminar, saia do prompt do MySQL:

```
EXIT;
```

MÓDULO IV

Pronto! Você agora criou um novo usuário **MySQL** e gerenciou suas permissões via linha de comando.

Isso oferece um controle preciso sobre quem pode fazer o quê em seus bancos de dados.

Lembre-se sempre de manter suas credenciais seguras e conceder apenas as permissões necessárias para garantir a segurança dos seus dados.

MÓDULO IV

EXERCÍCIOS...

MÓDULO IV

FIM...